# 02 - 模型架构面试题

> 共 28 道题，涵盖 Transformer 基础、注意力机制、位置编码、归一化、激活函数、FFN、残差连接、架构选择等核心知识点。

---

## Q1: Transformer 的整体架构是什么样的？Encoder 和 Decoder 分别做什么？

**详细版：**

原始 Transformer（Vaswani et al., 2017）由 Encoder 和 Decoder 两部分组成。Encoder 由 N 个相同的层堆叠而成，每层包含一个多头自注意力（Multi-Head Self-Attention）和一个前馈网络（FFN），每个子层后接残差连接和 LayerNorm。Decoder 也由 N 层组成，除了自注意力和 FFN 之外，还多了一个交叉注意力（Cross-Attention）层，用于关注 Encoder 的输出。Decoder 的自注意力使用 Causal Mask 来确保位置 i 只能看到位置 ≤i 的信息，保证自回归生成的因果性。

在 LLM 时代，GPT 系列采用了 Decoder-Only 架构，去掉了 Encoder 和 Cross-Attention，只保留了带 Causal Mask 的自注意力和 FFN。MiniMind 就是这种 Decoder-Only 架构。

**精简版：**

原始 Transformer 有 Encoder（双向注意力，适合理解任务）和 Decoder（因果注意力，适合生成任务）。LLM 普遍采用 Decoder-Only 架构，只保留 Causal Masked Self-Attention + FFN。MiniMind 是 Decoder-Only。

---

## Q2: Self-Attention 的计算公式是什么？请推导整个过程。

**详细版：**

给定输入 $X \in R^{n\times d}$（n 为序列长度，d 为维度），Self-Attention 的计算过程如下：

1. **线性投影**：$Q = XW_Q, K = XW_K, V = XW_V$，其中 $W_Q, W_K, W_V \in R^{d\times d_k}$
2. **计算注意力分数**：$Score = QK^T / \sqrt{d_k}$，得到 $n\times n$ 的矩阵
3. **Softmax 归一化**：$Attn = softmax(Score)$，每行归一化为概率分布
4. **加权求和**：$Output = Attn \cdot V$

完整公式：Attention(Q, K, V) = softmax(QK^T / √d_k) · V

直觉理解：Q 是「查询」，K 是「键」，V 是「值」。通过 Q 和 K 的点积计算相关度，然后用相关度对 V 做加权求和，相当于从序列中动态地提取与当前位置最相关的信息。

**精简版：**

Attention(Q,K,V) = softmax(QK^T/√d_k)·V。Q/K/V 由输入经线性投影得到，QK^T 计算 token 间的相关度，除以 √d_k 防止梯度消失，softmax 归一化后对 V 加权求和。

---

## Q3: 为什么要除以 √d_k?

**详细版：**

当 d_k 较大时，Q 和 K 的点积 QK^T 的值会随着维度增大而增大。假设 Q 和 K 的各分量是独立的均值为 0、方差为 1 的随机变量，那么 q·k = Σ(q_i · k_i) 的方差为 d_k。点积的量级约为 O(√d_k)，这会导致 softmax 的输入值过大，使得 softmax 函数进入梯度极小的饱和区（输出接近 one-hot），反向传播时梯度接近 0，训练困难。

除以 √d_k 后，点积的方差被缩放回 1，softmax 的输入维持在合理范围，梯度能正常回传。这就是所谓的 **Scaled Dot-Product Attention**。

**精简版：**

d_k 越大，QK^T 的点积值方差越大（方差 = d_k），会把 softmax 推入饱和区导致梯度消失。除以 √d_k 使方差恢复到 1，保持 softmax 梯度稳定。

---

## Q4: 什么是多头注意力（Multi-Head Attention）？为什么需要多头?

**详细版：**

多头注意力将 Q、K、V 分别拆成 h 个头，每个头独立计算注意力，最后拼接起来再过一个线性投影：

MultiHead(Q,K,V) = Concat(head_1, ..., head_h) · W_O 其中 head_i = Attention(QW_Q^i, KW_K^i, VW_V^i)

每个头的维度为 d_k = d_model / h。

**为什么需要多头？** 1. **捕获不同子空间的信息**：不同的头可以关注不同类型的关系（如语法关系、语义关系、位置关系等） 2. **增强表示能力**：单头注意力在一个子空间中只能计算一种注意力模式，多头可以同时学习多种模式 3. **计算效率**：总计算量与单头相当（每个头的维度缩小了 h 倍），但表示能力更强

MiniMind 使用 8 个注意力头，d_model=768，每个头的维度为 768/8=96。

**精简版：**

多头注意力将 Q/K/V 拆成 h 个头独立计算再拼接，每个头在不同子空间捕获不同的注意力模式。总计算量与单头相当，但表示能力更强。MiniMind 用 8 头，每头维度 96。

---

## Q5: 什么是 GQA？它与 MHA、MQA 有什么区别？

**详细版：**

三种注意力变体的核心区别在于 KV 头的数量：

| 机制 | Q 头数 | KV 头数 | 特点 |
|---|---|---|---|
| **MHA**（Multi-Head Attention） | h | h | 每个 Q 头配一对独立的 KV 头 |
| **MQA**（Multi-Query Attention） | h | 1 | 所有 Q 头共享同一对 KV 头 |
| **GQA**（Grouped Query Attention） | h | g（1<g<h） | 每组 h/g 个 Q 头共享一对 KV 头 |

GQA 是 MHA 和 MQA 的折中方案： - 相比 MHA：KV 缓存减少为原来的 g/h，推理时显存占用更低，速度更快 - 相比 MQA：保留了更多的 KV 容量，模型质量损失更小 - 当 g=h 时退化为 MHA，g=1 时退化为 MQA

MiniMind 配置 q_heads=8, kv_heads=4，即每 2 个 Q 头共享 1 对 KV 头，KV 缓存减半。

实现上，需要通过 `repeat_kv` 函数将 KV 头复制扩展到与 Q 头数量一致后再做注意力计算。

**精简版：**

GQA 是 MHA 和 MQA 的折中：MHA 每个 Q 头配独立 KV，MQA 所有 Q 头共享一套 KV，GQA 分组共享。MiniMind 用 8 个 Q 头、4 个 KV 头，KV 缓存减半，质量损失小。实现时用 repeat_kv 将 KV 复制到与 Q 头数对齐。

---

## Q6: RoPE（旋转位置编码）的原理是什么？

**详细版：**

RoPE 的核心思想是：将位置信息编码为旋转操作，使得两个 token 的注意力分数只取决于它们的相对位置。

具体做法：将 d 维向量两两分组，每一组看作一个二维平面上的向量，对其施加一个与位置 m 相关的旋转角度 θ_i：

f(x_m, m) 在第 i 组的效果是将 (x_{2i}, x_{2i+1}) 旋转角度 mθ_i

其中 θ_i = 1 / (base^{2i/d})，base 即 rope_theta。

**关键性质**：旋转后的 Q_m 和 K_n 的点积 只依赖于 (q, k, m-n)，即只依赖相对位置 m-n，满足相对位置编码的要求。

**实现方式**：利用复数乘法。将 (x_{2i}, x_{2i+1}) 视为复数 x_{2i} + ix_{2i+1}，乘以 e^{imθ_i} 即完成旋转。在实际代码中，等价于：

```
x_rot = x * cos(mθ) + rotate_half(x) * sin(mθ)
```

MiniMind 设置 rope_theta=1e6（而非默认的 10000），更大的 base 使高频分量衰减更慢，有利于长序列建模。

**精简版：**

RoPE 将位置编码为向量的旋转操作，使 QK 点积自然包含相对位置信息。将向量分量两两配对看作复数，乘以 e^{imθ} 完成旋转，θ_i = 1/base^{2i/d}。MiniMind 的 base=1e6，比默认 10000 更大，有利于长序列外推。

---

## Q7: RoPE 相比 Sinusoidal 位置编码有什么优势？

**详细版：**

| 特性 | Sinusoidal PE | RoPE |
| --- | --- | --- |
| 编码方式 | 加在输入 Embedding 上 | 乘在 Q/K 上（旋转） |
| 位置信息 | 绝对位置 | 自然编码相对位置 |
| 外推能力 | 差，超过训练长度急剧下降 | 更好，且可通过 NTK/YaRN 增强 |

| 特性 | Sinusoidal PE | RoPE |
| --- | --- | --- |
| 是否可学习 | 否（固定）/ 可改为可学习 | 否（固定公式） |
| 适用场景 | 原始 Transformer | 现代 LLM（LLaMA/Qwen/MiniMind） |

RoPE 的核心优势： 1. **相对位置编码**：注意力分数直接反映相对距离 2. **外推能力强**：通过调整 base（NTK-aware）或 YaRN 方法可以将上下文外推到更长 3. **与注意力耦合**：直接作用在 Q/K 上，不占用模型容量

**精简版：**

Sinusoidal PE 加在输入上编码绝对位置；RoPE 旋转 Q/K 编码相对位置。RoPE 外推能力更强，可通过 YaRN 扩展上下文长度，是现代 LLM 的主流方案。

---

## Q8: RMSNorm 和 LayerNorm 有什么区别？为什么 MiniMind 用 RMSNorm?

**详细版：**

**LayerNorm**：对每个样本的所有特征维度计算均值 μ 和方差 $\sigma^2$，然后标准化：

```
LayerNorm(x) = γ · (x - μ) / √(σ² + ε) + β
```

**RMSNorm**（Root Mean Square Normalization）：去掉了减均值（re-centering）操作，只做缩放（re-scaling）：

```
RMSNorm(x) = γ · x / √(mean(x²) + ε)
```

其中 $RMS(x) = \sqrt{(mean(x^2))}$。

**为什么用 RMSNorm？** 1. **计算效率**：省去了均值计算和减均值操作，减少约 10-15% 的计算量 2. **效果相当**：论文研究表明 re-centering 不是关键，re-scaling 才是核心 3. **实践验证**：LLaMA、Qwen 等主流模型都使用 RMSNorm，效果与 LayerNorm 相当 4. **参数更少**：没有偏置项 β

**精简版：**

LayerNorm = 减均值 + 除标准差 + 缩放偏移；RMSNorm 去掉减均值，只用 RMS 缩放。RMSNorm 计算更快（省约 10-15%），效果相当，LLaMA/Qwen/MiniMind 都在用。

---

## Q9: Pre-Norm 和 Post-Norm 有什么区别？为什么现代 LLM 偏好 Pre-Norm？

**详细版：**

**Post-Norm**（原始 Transformer）：

```
x = x + Sublayer(LayerNorm(x))  ← 错误写法
x = LayerNorm(x + Sublayer(x))  ← 正确：先子层，再残差，最后归一化
```

**Pre-Norm**（现代 LLM）：

```
x = x + Sublayer(Norm(x))  ← 先归一化，再子层，最后残差
```

**Pre-Norm 的优势：** 1. **训练更稳定**：梯度通过残差连接直接回传，不会被 Norm 层阻碍 2. **不依赖 warmup**：Post-Norm 通常需要精心设计的 warmup 策略 3. **可以训练更深的模型**：梯度路径更平滑

**Pre-Norm 的代价：** - 有理论分析指出 Pre-Norm 可能在表示能力上略弱于 Post-Norm（因为残差连接未经归一化的特征直接累加），但在实践中差距很小

MiniMind 使用 Pre-Norm + RMSNorm，每层的结构为：

```
x = x + Attention(RMSNorm(x))
x = x + FFN(RMSNorm(x))
```

**精简版：**

Post-Norm：子层输出加残差后再归一化；Pre-Norm：先归一化再进子层。Pre-Norm 训练更稳定、梯度回传更顺畅、可训练更深模型。MiniMind 用 Pre-Norm + RMSNorm。

---

## Q10: SwiGLU 激活函数是什么？为什么优于 ReLU/GELU？

**详细版：**

**SwiGLU** 是 Swish 激活函数与 GLU（Gated Linear Unit）的结合：

```
SwiGLU(x) = Swish(xW₁) ⊙ (xW₂)
其中 Swish(x) = x · sigmoid(βx)，通常 β=1
```

相比传统 FFN 的 $ReLU(xW_1)W_2$，SwiGLU 的特点是： 1. **门控机制**：GLU 引入了一个门控分支（$xW_2$），让网络学会选择性地传递信息 2. **平滑非线性**：Swish 比 ReLU 平滑，没有「死神经元」问题 3. **更强的表达能力**：实验表明在相同参数量下 SwiGLU 优于 ReLU 和 GELU

**注意**：SwiGLU 的 FFN 有三个权重矩阵（$W_1, W_2, W_3$），而标准 FFN 只有两个。为了保持参数量一致，通常将中间维度从 4d 调整为 8d/3 ≈ 2.67d（再取到合适的整数）。

MiniMind 的 FFN 结构：

```
self.gate_proj = nn.Linear(d_model, hidden_dim)   # W₁ (gate)
self.up_proj = nn.Linear(d_model, hidden_dim)      # W₂ (up)
self.down_proj = nn.Linear(hidden_dim, d_model)    # W₃ (down)

# forward:
output = self.down_proj(F.silu(self.gate_proj(x)) * self.up_proj(x))
```

**精简版：**

$SwiGLU(x) = Swish(xW_1) \odot (xW_2)$，结合 Swish 的平滑非线性和 GLU 的门控机制。比 ReLU/GELU 表达能力更强，无死神经元问题。因为有三个权重矩阵，中间维度通常取 8d/3 保持参数量不变。

---

## Q11: FFN 的扩展比（expansion ratio）通常是多少？为什么？

**详细版：**

在标准 Transformer 中，FFN 的中间维度 d_ff = 4 × d_model，扩展比为 4。

使用 SwiGLU 后，因为多了一个门控投影矩阵，为了保持总参数量一致，中间维度调整为：

```
d_ff = 8/3 × d_model ≈ 2.67 × d_model
```

实践中通常向上取整到某个方便的数（如 64 或 256 的倍数）。

**为什么需要扩展？** - 注意力层负责 token 间的信息交互 - FFN 负责对每个 token 独立地做非线性变换，是模型知识存储的主要位置 - 扩展到更高维度再投影回来，相当于在更大的空间中做变换，增强表示能力

**精简版：**

标准 FFN 扩展比为 4（d_ff = 4d）。SwiGLU 因多一个门控矩阵，调整为 8d/3 ≈ 2.67d 以保持参数量。FFN 是知识存储的主要位置，高维空间变换增强表示能力。

## Q12: 残差连接（Residual Connection）的作用是什么？

**详细版：**

残差连接的形式为 output = x + F(x)，即子层的输出加上原始输入。

**作用：** 1. **缓解梯度消失**：反向传播时，梯度可以通过残差捷径（shortcut）直接回传，不需要经过所有子层，使深层网络可以训练 2. **保证恒等映射**：如果某层不需要做变换，F(x) 学成 0 即可，网络至少不会比浅层差 3. **促进特征复用**：底层特征可以直接传递到高层

**在 Transformer 中**：每个注意力子层和 FFN 子层都有残差连接：

```
x = x + Attention(Norm(x))
x = x + FFN(Norm(x))
```

**精简版：**

残差连接 y = x + F(x) 让梯度可以跳过子层直接回传，缓解梯度消失，保证深层网络至少不比浅层差。Transformer 每个子层（Attention/FFN）都有残差连接。

---

## Q13: Decoder-Only 和 Encoder-Decoder 架构各适合什么任务？为什么 LLM 主流是 Decoder-Only？

**详细版：**

| 架构 | 代表模型 | 注意力方式 | 适合任务 |
|---|---|---|---|
| Encoder-Only | BERT | 双向注意力 | 理解任务（分类、NER） |
| Encoder-Decoder | T5, BART | 编码双向 + 解码因果 | 翻译、摘要等 seq2seq |
| Decoder-Only | GPT, LLaMA | 因果注意力 | 文本生成、LLM |

**Decoder-Only 成为 LLM 主流的原因：** 1. **统一的生成范式**：所有任务都可以转化为文本生成（prompt → completion） 2. **更好的 few-shot/zero-shot 能力**：因果语言模型天然适合 in-context learning 3. **架构简洁**：只有一种注意力机制，代码实现和推理优化更简单 4. **KV-Cache 友好**：因果注意力的 KV 可以缓存，加速自回归生成 5. **scaling law 更优**：有研究表明相同参数量下 Decoder-Only 的 scaling 效率更高

**精简版：**

Encoder-Decoder 适合 seq2seq 任务（翻译/摘要），Decoder-Only 适合生成任务。LLM 用 Decoder-Only 因为：统一生成范式、few-shot 能力强、架构简洁、KV-Cache 友好、scaling 更优。

---

## Q14: Embedding 层的作用是什么？什么是权重共享（Weight Tying）？

**详细版：**

**Embedding 层**（embed_tokens）将离散的 token ID 映射为连续的向量表示。它本质上是一个查找表（lookup table），形状为 (vocab_size, d_model)。

**LM Head 层**（lm_head）将模型最后一层的隐状态投影回词表维度，输出每个 token 的 logits，形状也是 (d_model, vocab_size)。

**权重共享（Weight Tying）**：让 embed_tokens 和 lm_head 共享同一个权重矩阵。

**好处：** 1. **减少参数量**：vocab_size × d_model 的参数只需要存一份。MiniMind 的词表 6400，d_model=768，共享节省了 6400×768 ≈ 4.9M 参数 2. **语义一致性**：输入和输出空间使用相同的表示，语义更一致 3. **正则化效果**：共享权重相当于隐式正则化，有助于防止过拟合 4. **在小模型中效果显著**：MiniMind 只有 64M 参数，节省的 4.9M 参数占比约 7.6%

**精简版：**

Embedding 将 token ID 映射为向量，lm_head 将隐状态映射回词表。权重共享让二者用同一个矩阵，节省参数（MiniMind 省约 4.9M/7.6%），并使输入输出语义一致。

---

## Q15: Causal Mask（因果掩码）的作用是什么？如何实现？

**详细版：**

Causal Mask 确保在计算注意力时，位置 i 只能关注位置 ≤i 的 token（即只能看到过去和当前，不能看到未来）。

**为什么需要？** - 自回归生成时，token 按顺序逐个生成，生成位置 i 的 token 时不应看到位置 i+1 及之后的信息 - 训练时通过 teacher forcing 并行计算所有位置的 loss，Causal Mask 保证每个位置的预测只依赖于之前的上下文

**实现方式**：构造一个上三角全为 -inf 的 mask 矩阵，加到注意力分数上：

```
mask = torch.triu(torch.ones(n, n), diagonal=1) * float('-inf')
attn_scores = attn_scores + mask
# softmax 后 -inf 对应的位置权重为 0
```

或者使用布尔 mask：

```
mask = torch.tril(torch.ones(n, n)).bool()
attn_scores.masked_fill_(~mask, float('-inf'))
```

**精简版：**

Causal Mask 是一个下三角矩阵，让位置 i 只能注意到 ≤i 的 token。实现上是在 attention score 的上三角部分加 -inf，softmax 后权重为 0。保证训练和推理的因果一致性。

---

## Q16: MiniMind 的参数量是怎么计算的?

**详细版：**

MiniMind 配置：d_model=768, n_layers=8, vocab_size=6400, q_heads=8, kv_heads=4, d_head=96, d_ff（SwiGLU 中间维度）

**各部分参数量：**

1. **Embedding**：vocab_size × d_model = 6400 × 768 = 4,915,200（约 4.9M） （权重共享，lm_head 不额外计算）

2. **每层 Attention**：

3. W_Q: d_model × (q_heads × d_head) = 768 × 768 = 589,824
4. W_K: d_model × (kv_heads × d_head) = 768 × 384 = 294,912
5. W_V: d_model × (kv_heads × d_head) = 768 × 384 = 294,912
6. W_O: d_model × d_model = 768 × 768 = 589,824

7. 小计：≈ 1.77M

8. **每层 FFN（SwiGLU）**：

9. gate_proj: d_model × d_ff
10. up_proj: d_model × d_ff
11. down_proj: d_ff × d_model

12. 小计：3 × d_model × d_ff

13. **每层 Norm**：2 × d_model（两个 RMSNorm，只有 γ 参数）

14. **Final Norm**：d_model = 768

总参数量 ≈ Embedding + 8 × (Attention + FFN + Norm) + Final Norm ≈ 64M

**精简版：**

参数分布：Embedding(6400×768≈4.9M) + 8层×(Attention≈1.77M + FFN + Norm) + Final Norm。Embedding 和 lm_head 共享。GQA 使 KV 投影只有 Q 的一半大小。总计约 64M。

---

## Q17: Softmax 的数值稳定性问题是什么？如何解决？

**详细版：**

Softmax(x_i) = exp(x_i) / Σ exp(x_j)

当 x_i 很大时，exp(x_i) 会溢出（overflow）；当 x_i 很小（负数很大）时，exp(x_i) 下溢为 0，分母可能为 0。

**解决方法**：减去最大值（safe softmax）：

```
Softmax(x_i) = exp(x_i - max(x)) / Σ exp(x_j - max(x))
```

数学上等价（分子分母同乘常数），但数值上稳定。PyTorch 的 `F.softmax` 内部已经实现了这个技巧。

在 Attention 中，Causal Mask 用 -inf 而非一个很大的负数，因为 exp(-inf)=0 是精确的。但如果某行全是 -inf（比如 padding），softmax 输出会变成 NaN，需要特殊处理。

**精简版：**

exp(x) 在 x 很大时会溢出。解决方法：softmax 前减去 max(x)，数学等价但数值稳定。PyTorch 已内置此优化。Mask 用 -inf 而非大负数因为 exp(-inf) 精确为 0。

---

## Q18: Transformer 中为什么用点积注意力而不是加性注意力？

**详细版：**

**加性注意力**（Bahdanau Attention）：

```
score(q, k) = v^T · tanh(W_q · q + W_k · k)
```

需要额外的参数 v 和 W_q, W_k，且 tanh 运算较慢。

**点积注意力**（Dot-Product Attention）：

```
score(q, k) = q^T · k / √d_k
```

直接利用矩阵乘法计算，GPU 上非常高效。

虽然加性注意力在小维度下可能效果更好（因为多了一层非线性），但在高维度下点积注意力的效率优势非常明显，且加上 scale 后效果不输加性注意力。

**精简版：**

点积注意力 q^T・k 直接用矩阵乘法完成，GPU 上极为高效；加性注意力需要额外参数和 tanh 非线性，效率低。高维度下缩放点积注意力效果不输加性注意力。

---

## Q19: 什么是 KQV 的维度关系？MiniMind 中是怎么配置的？

**详细版：**

标准 MHA 中： - d_model：模型隐藏维度 - n_heads：注意力头数 - d_head = d_model / n_heads：每个头的维度

Q 的总维度 = n_heads × d_head = d_model K 的总维度 = n_heads × d_head = d_model V 的总维度 = n_heads × d_head = d_model

在 GQA 中： - Q 的总维度 = q_heads × d_head - K 的总维度 = kv_heads × d_head - V 的总维度 = kv_heads × d_head

**MiniMind 配置**： - d_model = 768, q_heads = 8, kv_heads = 4, d_head = 768/8 = 96 - Q 维度：8 × 96 = 768 - K/V 维度：4 × 96 = 384 - W_Q 形状：(768, 768), W_K/W_V 形状：(768, 384)

**精简版：**

MiniMind：d_model=768, d_head=96。GQA 下 Q 是 8×96=768，K/V 是 4×96=384。W_Q 为 768×768，W_K/W_V 为 768×384。

---

## Q20: 模型的深度（层数）和宽度（隐藏维度）应该怎么权衡?

**详细版:**

在固定参数量的前提下，增加深度意味着减少宽度，反之亦然。

**深度优先的观点（MobileLLM 的发现）**： - MobileLLM 论文发现，在小模型（≤1B 参数）中，增加深度比增加宽度更有效 - 更深的模型能学到更复杂的层次化特征 - 建议：小模型应该「瘦长」而非「矮胖」

**宽度的优势**： - 更大的隐藏维度意味着每层的表示能力更强 - 过深的模型可能遇到训练不稳定问题 - 宽模型推理时并行度更高，硬件利用率更好

**MiniMind 的选择**：d_model=768, n_layers=8，属于适中的配置。对于 64M 参数量级，这个 8 层的设计可能还可以尝试更深更窄的配置（如 12 层 × 更小的 d_model）。

**精简版:**

MobileLLM 发现小模型中深度比宽度重要（「瘦长」优于「矮胖」）。但过深可能训练不稳定，且宽模型硬件并行度更高。MiniMind 用 d_model=768, 8 层，是一个折中。

---

## Q21: 什么是 Attention Sink 现象?

**详细版:**

Attention Sink 指的是 Transformer 模型倾向于给序列的第一个 token（通常是 BOS token 或第一个实际 token）分配异常高的注意力权重，即使这个 token 与当前位置的语义关系并不强。

**原因**： - Causal Mask 导致所有位置都能看到第一个 token，它成了一个"全局可见的锚点" - Softmax 的性质要求权重必须归一化到 1，当模型不需要关注任何特定 token 时，多余的注意力权重"倾倒"在第一个 token 上 - 第一个 token 的 KV 在所有位置的注意力计算中都参与，相当于一个隐式的"垃圾桶"

**影响**： - 推理时如果使用 sliding window 丢弃了第一个 token，性能会急剧下降 - StreamingLLM 的解决方案：始终保留前几个 token 的 KV-Cache

**精简版:**

Attention Sink 是模型倾向于给第一个 token 分配过高注意力的现象。因为 Causal Mask 下它对所有位置可见，成为注意力的"垃圾桶"。StreamingLLM 的解决方案是始终保留前几个 token 的 KV-Cache。

---

## Q22: MiniMind 对齐 Qwen3 是什么意思？具体对齐了哪些设计？

**详细版：**

"对齐 Qwen3"指的是 MiniMind 的模型架构设计参考了 Qwen3 的技术方案，具体包括：

1. **整体架构**：Decoder-Only Transformer
2. **注意力机制**：GQA（Grouped Query Attention）
3. **位置编码**：RoPE，支持 YaRN 外推
4. **归一化方案**：Pre-Norm + RMSNorm
5. **激活函数**：SwiGLU
6. **权重共享**：embed_tokens 和 lm_head 共享权重

MiniMind 的区别在于规模更小（64M vs Qwen3 的数十亿/数百亿参数），词表更小（6400 vs 通常 100k+），层数和维度也相应缩小。但核心架构设计完全一致。

**面试价值**：说「对齐 Qwen3」而不是「自己设计了个架构」，能体现你对主流工业级 LLM 架构的理解，以及务实的工程态度。

**精简版：**

对齐 Qwen3 指采用相同的架构设计：Decoder-Only + GQA + RoPE + Pre-Norm + RMSNorm + SwiGLU + 权重共享。MiniMind 是缩小版（64M 参数、6400 词表），核心设计一致。

---

## Q23: 为什么 Transformer 需要位置编码？不加会怎样？

**详细版：**

Self-Attention 的计算是**置换不变的**（permutation invariant）：对输入序列中 token 的任意排列，如果不加位置编码，注意力权重和输出不会改变。这意味着模型无法区分「猫吃鱼」和「鱼吃猫」。

**不加位置编码的后果**： - 模型无法理解词序，退化为"词袋模型" - 在实际任务中，效果会严重下降

**位置编码的演进**： 1. 固定 Sinusoidal PE（原始 Transformer） 2. 可学习的绝对位置编码（GPT-2） 3. 相对位置编码（T5 的 relative bias） 4. 旋转位置编码 RoPE（LLaMA/Qwen/MiniMind） 5. ALiBi（BLOOM）

MiniMind 使用 RoPE，将位置信息融入 Q/K 的旋转中。

**精简版：**

Self-Attention 是置换不变的，没有位置编码就无法区分词序（"猫吃鱼"="鱼吃猫"）。MiniMind 用 RoPE 将位置编码为 Q/K 的旋转操作。

---

## Q24: Transformer 的计算复杂度是多少？瓶颈在哪里？

**详细版：**

设序列长度为 n，隐藏维度为 d：

| 组件 | 时间复杂度 | 空间复杂度 |
|---|---|---|
| Self-Attention (QK^T) | $O(n^2d)$ | $O(n^2)$ |
| Self-Attention (·V) | $O(n^2d)$ | $O(n^2)$ |
| FFN | $O(nd^2)$ 或 O(nd·d_ff) | O(d·d_ff) |

**瓶颈分析**： - 当 n < d 时（短序列），FFN 是瓶颈：$O(nd^2)$ - 当 n > d 时（长序列），Attention 是瓶颈：$O(n^2d)$，且 $n^2$ 的注意力矩阵显存也会爆

**优化方向**： - FlashAttention：减少 HBM 访问，不改变复杂度但减少实际运行时间 - 线性注意力（Linear Attention）：将 $O(n^2)$ 降为 O(n) - 稀疏注意力：只计算部分 token 对的注意力 - MoE：保持 Attention 不变，FFN 只激活部分专家

**精简版：**

Attention 复杂度 $O(n^2d)$，FFN 复杂度 $O(nd^2)$。短序列 FFN 是瓶颈，长序列 Attention 是瓶颈。优化方向：FlashAttention 减少内存访问，线性注意力降低到 O(n)，MoE 减少 FFN 计算。

---

## Q25: 什么是 Logits？Logits 是怎么变成概率和最终生成 token 的？

**详细版：**

**Logits** 是模型最后一层输出经过 lm_head 线性投影后得到的未归一化分数向量，维度为 vocab_size。每个值代表对应 token 的"置信度"。

**从 Logits 到概率**：

```
prob = softmax(logits / temperature)
```

temperature 控制分布的尖锐程度：T→0 趋近 greedy，T→∞ 趋近均匀分布。

**从概率到 token**（采样策略）： 1. **Greedy**：直接取 argmax(logits) 2. **Top-K**：只保留概率最高的 K 个 token，然后从中采样 3. **Top-P（Nucleus Sampling）**：保留累积概率达到 P 的最小 token 集合，然后采样 4. **Temperature + Top-P**：先调温度再 Top-P，综合控制

**训练时**：logits 直接送入 CrossEntropyLoss（内部包含 softmax），不需要手动转概率。

**精简版：**

Logits 是 lm_head 输出的未归一化分数（维度=vocab_size）。通过 softmax 转为概率，再用 greedy/top-k/top-p 采样得到最终 token。训练时直接用 CrossEntropyLoss（内含 softmax）。

---

## Q26: 为什么 Transformer 比 RNN/LSTM 更适合做大语言模型?

**详细版：**

| 特性 | RNN/LSTM | Transformer |
|---|---|---|
| 并行化 | 无法并行（依赖前一步的隐状态） | 完全并行（注意力矩阵同时计算） |
| 长距离依赖 | 信息传递需经过所有中间步骤，容易衰减 | 任意两个位置直接连接，O(1) 路径长度 |
| 训练速度 | 慢（序列依赖） | 快（GPU 并行优化） |
| 缩放能力 | 差（隐状态维度固定为瓶颈） | 好（Scaling Law，可通过增加参数提升性能） |
| 上下文长度 | 理论无限但实际有限 | 固定窗口但可通过外推技术扩展 |

**Transformer 替代 RNN 的根本原因**：GPU 并行计算的效率。Transformer 把序列建模转化为大矩阵乘法，充分利用 GPU 的并行能力。

**精简版：**

Transformer 可完全并行训练（vs RNN 必须顺序计算）、长距离依赖 O(1) 路径（vs RNN 信息逐步衰减）、Scaling Law 缩放性好。根本原因是 GPU 对矩阵乘法的并行优化。

---

## Q27: LayerNorm 和 BatchNorm 有什么区别？为什么 NLP 用 LayerNorm？

**详细版：**

| 特性 | BatchNorm | LayerNorm |
|---|---|---|
| 归一化维度 | 对 batch 维度归一化（同一特征跨样本） | 对 feature 维度归一化（同一样本内所有特征） |
| batch 依赖 | 依赖 batch 统计量 | 不依赖 batch |
| 推理时 | 需要维护 running mean/var | 直接计算，无需额外统计 |
| 变长序列 | 不同样本长度不同，padding 导致统计不准 | 每个 token 独立归一化，不受影响 |

**NLP 用 LayerNorm 的原因**： 1. NLP 序列长度不固定，BatchNorm 的 batch 统计在变长序列上不可靠 2. 自回归生成时 batch_size 可能为 1，BatchNorm 无法工作 3. LayerNorm 对每个 token 独立归一化，不受 batch 和序列长度影响

**精简版：**

BatchNorm 跨 batch 归一化，依赖 batch 统计量；LayerNorm 在 feature 维度归一化，每个样本独立。NLP 用 LayerNorm 因为序列变长、batch 统计不可靠、推理时 batch=1 也能工作。

---

## Q28: 什么是 Perplexity（困惑度）？它和 Loss 什么关系？

**详细版：**

**Perplexity (PPL)** 是语言模型最常用的评估指标：

```
PPL = exp(H) = exp(-1/N × Σ log P(x_i | x_{<i}))
```

其中 H 是交叉熵（Cross-Entropy Loss），N 是 token 数。

**直觉理解**：PPL 可以理解为模型在每个位置的"平均选择数"。PPL=10 意味着模型每一步平均要在 10 个等可能的 token 中选择。PPL 越低越好。

**与 Loss 的关系**：PPL = exp(Loss) - Loss = 2.0 → PPL = e^2 ≈ 7.39 - Loss = 3.0 → PPL = e^3 ≈ 20.09

**注意**：PPL 只适用于同一数据集和同一词表的比较。不同词表大小、不同测试集的 PPL 不可直接比较。

**精简版：**

Perplexity = exp(CrossEntropyLoss)，表示模型每步的"平均选择数"，越低越好。PPL 只在相同数据集和词表下可比较。